

Assignment 2

Q.1.

Metamask account address:

0x4eeD2f098b1A61e931Bf9a2B3ce1f7b93264Eb00

Q.2.

The Ethereum Whitepaper introduces Ethereum as a revolutionary decentralized platform that goes beyond Bitcoin's original purpose of facilitating digital currency transactions. At its heart, Ethereum enables developers to build and run smart contracts—self-executing programs that follow predefined rules without needing intermediaries. This opens the door to creating a wide range of decentralized applications (dApps) that serve countless use cases.

The key idea is to generalize the blockchain concept: while Bitcoin is designed specifically for transferring currency, Ethereum provides a Turing-complete programming environment on its blockchain. This means developers can write programs with complex logic, bringing automation and transparency to processes like financial agreements, supply chain management, and governance.

Ethereum's Core Concepts:

1. State and Accounts:

Ethereum operates as a global system where every action updates the state.

It recognizes two types of accounts:

- Externally Owned Accounts (EOA): These are controlled by individuals via private keys, similar to Bitcoin wallets.
- Contract Accounts: These house smart contracts—pieces of code that execute based on incoming instructions (transactions or messages).

2. Transactions and Gas:

Transactions on Ethereum aren't free; they consume computational resources measured in gas. Users pay for gas using Ether (ETH), Ethereum's cryptocurrency. This system ensures resources are used efficiently and prevents spam or malicious activity by requiring users to bear the cost of their transactions.

3. Smart Contracts:

The real magic of Ethereum lies in its smart contracts. These are autonomous programs stored on the blockchain that can handle anything from managing financial derivatives to creating voting systems. They are transparent, tamper-proof, and execute exactly as programmed, fostering trust without requiring intermediaries.

4. Blockchain and Consensus:

Ethereum's blockchain is the public ledger that records all transactions and state changes. Initially, Ethereum used Proof of Work (PoW)—similar to Bitcoin—for securing its network. However, to address scalability and energy

efficiency concerns, Ethereum is transitioning to Proof of Stake (PoS), where validators stake Ether to propose and validate blocks.

5. Applications:

Ethereum's versatility has led to groundbreaking innovations:

- Decentralized Finance (DeFi) applications democratize financial services like lending and trading.
- Tokenization enables the creation of new cryptocurrencies and digital assets.
- Identity Management allows individuals to securely control their personal data.
- Governance Systems facilitate transparent and fair decision-making processes.

Ethereum's Vision for the Future:

The whitepaper paints Ethereum as the cornerstone of Web 3.0, a decentralized version of the internet where individuals regain control of their data and interactions. It envisions a world where innovation flourishes, and applications are built on trust, transparency, and collaboration. By giving developers the tools to create these decentralized systems, Ethereum aims to disrupt traditional industries, reduce dependence on intermediaries, and pave the way for a fairer digital economy. In essence, Ethereum is more than a blockchain; it's a platform for reimagining how we interact digitally, one that prioritizes empowerment, security, and inclusivity for all.

Q.3.

Contract Address:

0x53deBbA7F6772Fe7990437bF16A5739ad46747D5

Q.4.

A wallet is a tool that allows users to store, transfer, and manage their digital assets.

Software Wallets:

A crypto software wallet, similar to a mobile banking app for your cryptocurrency, is a program you install on your device that allows you to manage your crypto holdings on the blockchain. It doesn't store the actual cryptocurrency itself but holds the keys that grant you access to it on the network.

This lets you view your balance, send and receive crypto, and even interact with decentralized applications. Software wallets are known for their convenience and ease of use, making them a popular choice for beginners and experienced crypto users alike.

Hardware Wallets:

For maximum security of your cryptocurrency, consider a hardware wallet. Unlike software wallets that live on your computer or phone, hardware wallets are physical devices, often resembling USB drives. These keep your private keys offline,

eliminating the risk of online hacks. While less convenient than software wallets for everyday use due to the need to connect them to your device for transactions, hardware wallets are the ultimate solution for serious crypto holders who prioritize security.

A durable hardware wallet goes beyond just storing your crypto. It's built tough, with a strong casing of stainless steel or reinforced plastic to shrug off everyday wear and tear. If you are purchasing one, look for established brands known for their secure products, as they prioritize quality materials and construction.

Wallets don't break the principle of decentralization because they primarily act as interfaces to interact with decentralized systems, like blockchains, rather than centralized intermediaries. Here's why wallets align with decentralization:

How Wallets Preserve Decentralization

1. **Non-Custodial Nature:** Many wallets (e.g., Metamask) are non-custodial, meaning users maintain full control of their private keys and funds. The wallet is simply a tool for managing and signing transactions, not a service that takes custody of assets.
2. **Direct Blockchain Interaction:** Wallets allow users to directly interact with decentralized networks without relying on a central authority. Transactions are sent to and confirmed by the blockchain, not mediated by the wallet provider.
3. **Open Source:** Many wallets are open-source, ensuring transparency. Developers and users can audit the code to verify that wallets function as expected without hidden controls or backdoors.
4. **Decentralized Identity:** Wallets facilitate decentralized identity and data ownership. Users can control their keys, which is fundamental to maintaining sovereignty over assets and interactions.
5. **Optional Use:** The choice of wallet is entirely up to the user. With hundreds of wallets available, users are not tied to a single provider, promoting a competitive and decentralized ecosystem.

Alternatives to Metamask:

1. Trust Wallet
2. Coinbase Wallet
3. Ledger Live
4. Trezor Suite
5. Exodus Wallet
6. MyEtherWallet (MEW)
7. Rainbow Wallet
8. Argent Wallet

Q.5.

A "Zero address" in the context of cryptography, particularly blockchain technology, refers to a specific public address that is essentially "null" or unusable, and **the effort to obtain its corresponding private key is considered impossible** because it would require solving a computationally infeasible problem within the cryptographic system used to generate addresses; essentially, it's like trying to crack a very strong encryption with no known vulnerabilities.

Q.6.

Brave Browser is a web browser that prioritizes user privacy by blocking most online trackers and ads, and its native cryptocurrency is called Basic Attention Token (BAT), which users can earn by viewing privacy-focused ads within the browser through a feature called "Brave Rewards"; essentially, users are rewarded for their attention with BAT tokens when they choose to see selected ads on the platform.

Q.7.

The key difference between ERC-721 and ERC-1155 lies in their flexibility and efficiency. ERC-721 is designed specifically for managing unique, non-fungible tokens (NFTs), where each token is represented by a distinct ID. This makes ERC-721 ideal for applications like digital art or collectibles, where every item is unique. However, it lacks support for batch operations, meaning that transferring or minting multiple tokens requires individual transactions, which can be costly in terms of gas fees.

On the other hand, ERC-1155 is a multi-token standard that supports both fungible and non-fungible tokens within the same contract. This makes it more versatile, especially for use cases like gaming, where assets can range from unique items (e.g., rare swords) to fungible ones (e.g., in-game currency). ERC-1155 allows batch operations, enabling multiple tokens to be minted, transferred, or burned in a single transaction, significantly reducing gas costs

Q.8.

This program uses the eth-account library to generate a valid Ethereum address. So we first install the library:

```
!pip install eth-account
```

The program will:

- Generate a random 32-byte private key using the secrets module
- Create an Ethereum address from this private key
- Display both the address and private key

```
from eth_account import Account
import secrets
def generate_ethereum_address():
    private_key = secrets.token_hex(32)
    account = Account.from_key('0x' + private_key)
```

```

    return {
        'address': account.address,
        'private_key': private_key
    }

if __name__ == "__main__":
    Account.enable_unaudited_hdwallet_features()
    wallet = generate_ethereum_address()
    print("Ethereum Address:", wallet['address'])
    print("Private Key:", wallet['private_key'])

```

Sample Output:

Ethereum Address: 0xb8aE50Aa4580a196e333acEe35a340cde2d0f6Ad

Private Key: 59c4dfcd2004c4ed12cc3da6cf40f6def21d67c7e89a3bcd1fc6a3c98d97473d

Q.9.

First I have to install this:

!pip install eth-utils rlp

Python code to generate a contract address using the sender's address and nonce. The code uses the eth_utils library for RLP encoding and Keccak-256 hashing.

```

from eth_utils import keccak, to_checksum_address
from rlp import encode

def generate_contract_address(sender_address, nonce):
    sender_bytes = bytes.fromhex(sender_address[2:]) # Convert sender address to bytes
    rlp_encoded = encode([sender_bytes, nonce]) # RLP encode sender and nonce
    contract_hash = keccak(rlp_encoded) # Compute Keccak-256 hash
    return to_checksum_address(contract_hash[-20:]) # Return last 20 bytes as address

# Hardcoded inputs
sender_address = "0x1234567890abcdef1234567890abcdef12345678"
nonce = 1

# Generate and print the contract address
print("Sender Address:", sender_address)
print("Nonce:", nonce)
print("Generated Contract Address:", generate_contract_address(sender_address, nonce))

```

Output:

Sender Address: 0x1234567890abcdef1234567890abcdef12345678

Nonce: 1

Generated Contract Address: 0x604f1ECbA68f4B4Da57D49C2b945A75bAb331208

Q.10.

First we have to install the library:

!pip install web3 eth-account

Code in python:

```
from web3 import Web3
from eth_account import Account
import secrets

class EthereumTransactionHandler:
    def __init__(self):
        self.web3 = Web3()

    def get_address_from_private_key(self, private_key):
        if not private_key.startswith('0x'):
            private_key = '0x' + private_key
        account = Account.from_key(private_key)
        return account.address

    def sign_transaction(self, recipient_address, ether_amount, private_key):
        if not self.web3.is_address(recipient_address):
            raise ValueError("Invalid recipient address")

        transaction = {
            'nonce': 0,
            'maxFeePerGas': self.web3.to_wei(50, 'gwei'),
            'maxPriorityFeePerGas': self.web3.to_wei(2, 'gwei'),
            'gas': 21000,
            'to': recipient_address,
            'value': self.web3.to_wei(ether_amount, 'ether'),
            'data': '0x',
            'chainId': 1,
            'type': '0x2'
        }

        if not private_key.startswith('0x'):
            private_key = '0x' + private_key
        return self.web3.eth.account.sign_transaction(transaction, private_key)

    def verify_transaction(self, signed_transaction, expected_sender):
        try:
            expected_sender = self.web3.to_checksum_address(expected_sender)
            recovered_address =
self.web3.eth.account.recover_transaction(signed_transaction.raw_transaction)
            return self.web3.to_checksum_address(recovered_address) == expected_sender
```

```
except Exception:  
    return False
```

```
def main():  
    transaction_handler = EthereumTransactionHandler()  
    private_key = secrets.token_hex(32)  
    sender_address = transaction_handler.get_address_from_private_key(private_key)  
    recipient_address = '0x742d35Cc6634C0532925a3b844Bc454e4438f44e'  
  
    try:  
        signed_transaction = transaction_handler.sign_transaction(recipient_address, 0.1,  
private_key)  
        is_valid = transaction_handler.verify_transaction(signed_transaction, sender_address)  
        print(f"Transaction hash: {signed_transaction.hash.hex()}")  
        print(f"Valid: {is_valid}")  
    except Exception as error:  
        print(f"Error: {error}")  
  
if __name__ == "__main__":  
    main()
```

Output:

Transaction hash:

d1aa6aaf559da3e5bfc538157dbee752fa6f7633c492808de2add606a5becf0c

Valid: True